

Directives pour Agents IA

Configuration du projet de Galerie d'Images - AGENTS.md (Version Corrigée)

Golden Rules – Règles Absolues

Ces règles ne doivent jamais être transgressées.

1. **Respecte DESIGN.md et ANTIGRAVITY.md à la lettre** : Toute modification d'interface, de composant ou de style doit être strictement conforme aux règles définies dans DESIGN.md. Les configurations d'environnement de ANTIGRAVITY.md priment sur le reste.
2. **Zéro chaîne de caractères hardcodée** : Aucun texte visible par l'utilisateur ne doit être écrit directement dans le code. Tout passe obligatoirement par le hook `useTranslation()` de `react-i18next`.
3. **Minimalisme extrême** : Priorise toujours un code fonctionnel, durable et facilement maintenable. Évite toute sur-ingénierie. Aucune nouvelle dépendance ne doit être ajoutée sans validation explicite.
4. **Demande avant d'improviser** : Si une fonctionnalité ou décision d'architecture n'est pas clairement documentée → pose la question au lieu de deviner.
5. **Respecte .editorconfig + langue dans le code** : Analyse et respecte impérativement les règles du fichier `.editorconfig`. Commentaires, commits et doc technique en anglais (sauf AGENTS.md et ANTIGRAVITY.md qui restent en français).

1. Stack Technologique Principal

- **Frontend** : React / Vite
- **Backend** : C# / ASP.NET Core
- **Base de données** : MariaDB

- **Conteneurisation & Déploiement** : Docker / Fly.io
- **Internationalisation** : react-i18next (FR, EN)

2. Conventions de Développement

Frontend (React & Vite)

- Utiliser exclusivement des composants fonctionnels et des Hooks.
- Les mises à jour de l'interface liées aux interactions doivent être optimistes pour garantir la fluidité.

Backend (C# .NET Core)

- Garder les contrôleurs légers. La logique métier doit être extraite dans des services indépendants et injectables.
- Maintenir une stricte intégrité référentielle dans MariaDB.
- Utiliser des standards cryptographiques robustes (SHA512).

3. Déploiement et Infrastructure

- Services déployés sur Fly.io via des conteneurs.
- Toute modification de dépendances ou variables (VITE_) doit être reflétée dans les Dockerfile.

4. Tests et Validation

- **Audit de l'existant** : Avant toute modification, vérifier systématiquement si des tests existent déjà pour le code ciblé (**xUnit** pour le backend, **Vitest** pour le frontend). Si la logique change, mettre à jour les tests existants.
- **Couverture des nouveautés** : Toute nouvelle fonctionnalité touchant à l'accès aux données doit être accompagnée de nouveaux tests appropriés.
- **Validation stricte** : S'assurer que l'ensemble de la suite de tests passe avec succès avant toute soumission.